Q0 Before you start; before you submit

I did not collaborate with anyone on this homework.

Q1 Self assessment

The part of HW4 with the most room for improvement is Question 3(b), the exchange argument for the greedy refueling algorithm. My answer was correct, but it was much longer and harder to read than the posted one. Here is the middle of what I submitted, after I defined j as the first town of OPT after $a = i_{\text{diff}-1}$ and set $\text{OPT}' = (\text{OPT} \setminus \{j\}) \cup \{i_{\text{diff}}\}$:

> OPT' *is valid. Let* $p < q$ *be two consecutive elements of* $\{0\} \cup \text{OPT}' \cup \{n\}$*; I show* $\text{dist}(p, q) \leq R$*. Note that* $i_{diff} \in \text{OPT}'$*, so it is impossible that* $p < i_{diff} < q$*. That leaves two cases. Case* $q \leq i_{diff}$*. The town* a *is in* $\{0\} \cup \text{OPT}'$ *..., so* $a < q$*. Since* p *is the element right before* q*, we get* $p \geq a$*. So* $a \leq p < q \leq i_{diff}$*, and by Facts 1 and 2,* $\text{dist}(p, q) \leq \text{dist}(a, i_{diff}) \leq R$*. Case* $p \geq i_{diff}$*. Above* i_{diff}*, the sets* OPT *and* OPT' *agree ... If* $p > i_{diff}$*, then* p *and* q *are also consecutive in* $\{0\} \cup \text{OPT} \cup \{n\}$ *... If* $p = i_{diff}$*, then* q *is the smallest element of* $\text{OPT}' \cup \{n\}$ *above* i_{diff} *... Let* p' *be the element of* $\{0\} \cup \text{OPT}$ *right before* q *... so* $p' < i_{diff}$ *... Fact 1 gives* $\text{dist}(i_{diff}, q) \leq \text{dist}(p', q) \leq R$*.*

What was deficient: nothing is wrong logically, but the organization is bad. I proved validity by taking an arbitrary pair of consecutive stops $p < q$ in OPT' and splitting into cases on where the pair sits relative to i_{diff}. That forces the reader to track two helper facts, three cases, and extra names like p', just to check something that has a two-line explanation. If I were grading it I would have a hard time finding the three things that actually matter.

How to fix it: the posted solution organizes the same argument around the stretches of OPT' instead of around arbitrary pairs. There are only three kinds of stretches. (1) Every stretch up to and including the one that ends at i_{diff} lies inside a stretch that the greedy algorithm drove on one pack, so it has distance at most R. (2) The stretch leaving i_{diff} ends at the next stop of OPT and starts later than OPT's own stretch into that stop (because $j < i_{\text{diff}}$), so it is shorter than a stretch of OPT, hence at most R. (3) Every stretch after that is unchanged from OPT. Writing it this way, the proof fits in one short paragraph, and each sentence corresponds directly to one rubric item (the stretches into i_{diff}, the stretch out of i_{diff}, and the rest). Next time I will first identify what actually changes between OPT and OPT' and only argue about those parts, instead of re-checking everything from scratch.

Q2.1 Keeping things fresh with potpourri (a)

Recurrence. For $n \leq 2$ the function does a constant amount of work (one comparison and possibly a swap), so $T(1) = T(2) = \Theta(1)$. For $n > 2$ it computes $t = \lceil 2n/3 \rceil$, makes three recursive calls on subarrays of size t, and does nothing else. Passing a subarray just means passing two indices, nothing gets copied, so the work outside the recursive calls is $\Theta(1)$. Ignoring the ceiling, as allowed,

$$T(n) = 3\,T\left(\tfrac{2n}{3}\right) + \Theta(1) \qquad (n > 2), \qquad T(1) = T(2) = \Theta(1).$$

Master Theorem. Here $a = 3$, $b = 3/2$, and $f(n) = \Theta(1) = \Theta(n^0)$. The critical exponent is

$$\log_b a = \log_{3/2} 3 = \frac{\ln 3}{\ln(3/2)} \approx 2.71.$$

Since $f(n) = O(n^{\log_b a - \varepsilon})$ for, say, $\varepsilon = 1$, this is the first case of the Master Theorem, where the leaves of the recursion tree dominate, and

$$T(n) = \Theta\left(n^{\log_{3/2} 3}\right) \approx \Theta(n^{2.71}).$$

So StoogeSort is slower than even insertion sort.

Q2.2 Keeping things fresh with potpourri (b)

There is no valid potential function, because ALG does not even halt on every valid input.

An input on which it runs forever. Take $a = 4$ and $b = 2$ (both powers of two).

- ALG$(4, 2)$: none of $a = 1$, $b = 1$, $a = b$ holds, and $a > b$, so it calls ALG$(4/2,\ 2{\cdot}2) =$ ALG$(2, 4)$.
- ALG$(2, 4)$: again no base case holds, and $b > a$, so it calls ALG$(2 \cdot 2,\ 4/2) =$ ALG$(4, 2)$.

So the computation is $(4, 2) \to (2, 4) \to (4, 2) \to (2, 4) \to \cdots$, and it never reaches a base case.

Why this rules out a potential function. A potential function has to be a non-negative integer that strictly decreases on every recursive call. On the run above, the pair of arguments $(4, 2)$ comes back after two calls. If Φ were a valid potential function, we would need $\Phi(4, 2) > \Phi(2, 4) > \Phi(4, 2)$, which is impossible. So no potential function can exist for this algorithm.

(What is going on: write $a = 2^i$ and $b = 2^j$. Every call keeps $i + j$ fixed and changes $i - j$ by exactly 2. When $i + j$ is odd, $i - j$ is odd, so it can never become 0, and once $|i - j| = 1$ the two calls just swap back and forth.)

Q2.3 Keeping things fresh with potpourri (c)

IsPrime is *not* efficient.

"Efficient" means running in time polynomial in the input size. The input is the integer k, written in binary (or in any base other than unary), so the input size is $n = \Theta(\log k)$ bits.

The loop runs for $i = 2, \ldots, \left\lfloor \sqrt{k} \right\rfloor$, so in the worst case (for example when k is prime, so the loop never returns early) it does about $\sqrt{k}$ iterations. In terms of the input size, $\sqrt{k} \approx \sqrt{2^n} = 2^{n/2}$. So the number of iterations alone is exponential in n. Even though each divisibility check takes only polynomial time in n, as we are allowed to assume, the total running time is at least $2^{n/2}$ steps, which is not bounded by any polynomial in n. For instance, a 200-bit prime would need about 2^{100} iterations.

So the algorithm is correct but not efficient with respect to the input size. (It would be polynomial only if k were written in unary, where the input size is k itself, but that is not the standard encoding.)

Q3.1 DFA warmup (a)

Reading the transitions off the diagram:

- $\Sigma = \{a, b\}$.
- $Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$.
- Start state: q_0.
- $F = \{q_4\}$ (the only state drawn with a double circle).
- Transition function $\delta : Q \times \Sigma \to Q$:

δ	a	b
q_0	q_3	q_2
q_1	q_3	q_5
q_2	q_2	q_2
q_3	q_1	q_2
q_4	q_2	q_5
q_5	q_2	q_4

Note that q_2 is a "dead" state: both of its transitions loop back to q_2, and it is not accepting, so once the DFA enters q_2 the input is rejected no matter what comes next.

Q3.2 DFA warmup (b)

Running the DFA on each string (the state sequence is shown for reference):

- ε: stays in q_0. Not accepted.
- ba: $q_0 \to q_2 \to q_2$. Not accepted.
- $aabb$: $q_0 \to q_3 \to q_1 \to q_5 \to q_4$. **Accepted.**
- $aabbbb$: $q_0 \to q_3 \to q_1 \to q_5 \to q_4 \to q_5 \to q_4$. **Accepted.**
- $aababba$: $q_0 \to q_3 \to q_1 \to q_5 \to q_2 \to q_2 \to q_2 \to q_2$. Not accepted.
- $aaaabb$: $q_0 \to q_3 \to q_1 \to q_3 \to q_1 \to q_5 \to q_4$. **Accepted.**
- $aaabbbb$: $q_0 \to q_3 \to q_1 \to q_3 \to q_2 \to q_2 \to q_2 \to q_2$. Not accepted.

The accepted strings are $aabb$, $aabbbb$, and $aaaabb$.

Q3.3 DFA warmup (c)

The DFA decides

$$L = (aa)^+(bb)^+ = \left\{a^{2i}b^{2j} : i \geq 1,\ j \geq 1\right\},$$

the strings consisting of a positive even number of a's followed by a positive even number of b's.

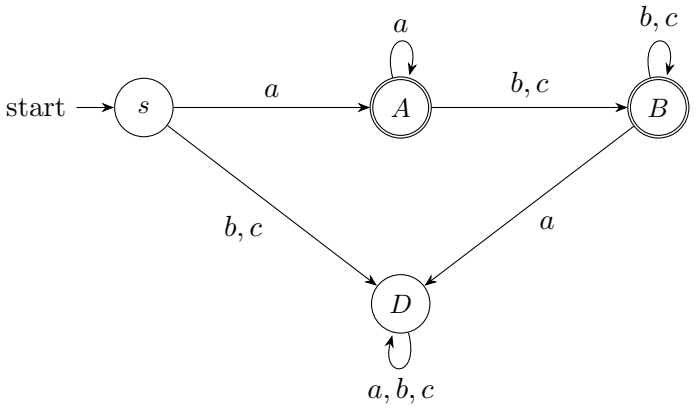
Justification. As noted in part (a), q_2 is a dead state, so any string that ever enters q_2 is rejected. Now follow the structure of the diagram.

- From q_0, reading b goes to q_2 (dead). So an accepted string must start with a.
- The a's alternate between q_3 and q_1: after an odd number of leading a's the DFA is in q_3, after an even number (at least 2) it is in q_1. Reading b in q_3 goes to q_2 (dead), while reading b in q_1 goes to q_5. So the block of leading a's must have even length, at least 2, and it must be followed by a b (ending in q_1 or q_3 is not accepting).
- Once in q_5 or q_4, reading a goes to q_2 (dead), so no more a's are allowed. The b's alternate between q_5 and q_4: after an odd number of b's the DFA is in q_5, after an even number it is in q_4. Only q_4 is accepting, so the number of b's must be even and at least 2.

Putting this together, the DFA ends in q_4 exactly on strings of the form $a^{2i}b^{2j}$ with $i, j \geq 1$, which is $(aa)^+(bb)^+$.

Q4.1 Constructing DFAs (a)

The language is $a^+(b \mid c)^*$: one or more a's, followed by any string over $\{b, c\}$. The following DFA with 4 states decides it.



Formally, $\Sigma = \{a, b, c\}$, $Q = \{s, A, B, D\}$, start state s, $F = \{A, B\}$, and

δ	a	b	c
s	A	D	D
A	A	B	B
B	D	B	B
D	D	D	D

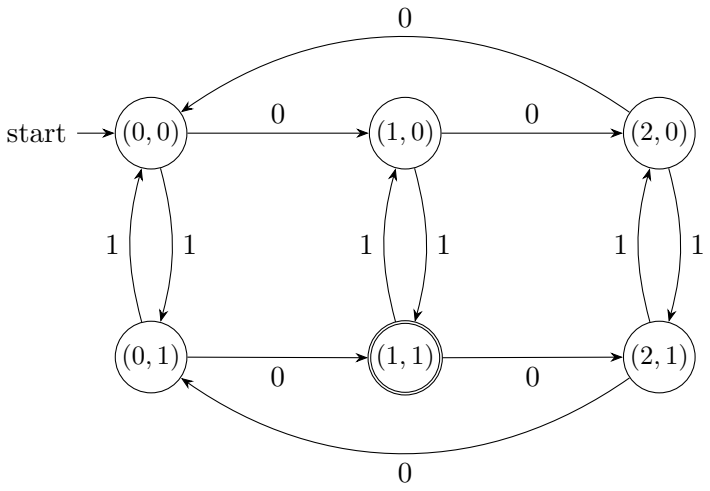
Justification. Each state records what has been read so far:

- s: nothing read yet.
- A: the input so far is a^+ (one or more a's and nothing else). This is in the language, so A accepts.
- B: the input so far is $a^+(b \mid c)^+$. This is in the language, so B accepts.
- D: the input so far cannot be extended to a string in the language, either because it started with b or c, or because an a appeared after some b or c. D is a dead state.

Checking that the transitions keep these descriptions true: from s, an a gives a^+ and anything else is dead; from A, another a keeps a^+ and a b or c moves to $a^+(b|c)^+$; from B, more b's and c's stay in B, while an a breaks the pattern. A string is in $a^+(b|c)^*$ exactly when it is of the form a^+ or $a^+(b|c)^+$, i.e., exactly when the DFA ends in A or B.

Q4.2 Constructing DFAs (b)

The DFA has to remember two things at once: the number of 0's mod 3 and the number of 1's mod 2. So I use $3 \cdot 2 = 6$ states named (i, j), where $i \in \{0, 1, 2\}$ is the number of 0's read so far mod 3 and $j \in \{0, 1\}$ is the number of 1's read so far mod 2. Reading a 0 moves $i \mapsto (i + 1) \bmod 3$ and leaves j alone; reading a 1 flips j and leaves i alone.



Formally, $\Sigma = \{0, 1\}$, $Q = \{0, 1, 2\} \times \{0, 1\}$, start state $(0, 0)$, $F = \{(1, 1)\}$, and

$$\delta\big((i, j), 0\big) = \big((i + 1) \bmod 3,\ j\big), \qquad \delta\big((i, j), 1\big) = \big(i,\ (j + 1) \bmod 2\big).$$

Justification. Claim: after reading any string w, the DFA is in state $(\#_0(w) \bmod 3,\ \#_1(w) \bmod 2)$, where $\#_0(w)$ and $\#_1(w)$ count the 0's and 1's in w. This holds for $w = \varepsilon$ because the start state is $(0, 0)$. If it holds for w, then appending a 0 increases $\#_0$ by one and leaves $\#_1$ alone, which is exactly what the 0-transition does to the state, and similarly appending a 1 flips the parity of $\#_1$ and leaves $\#_0$ alone. So the claim holds for all w by induction on $|w|$.

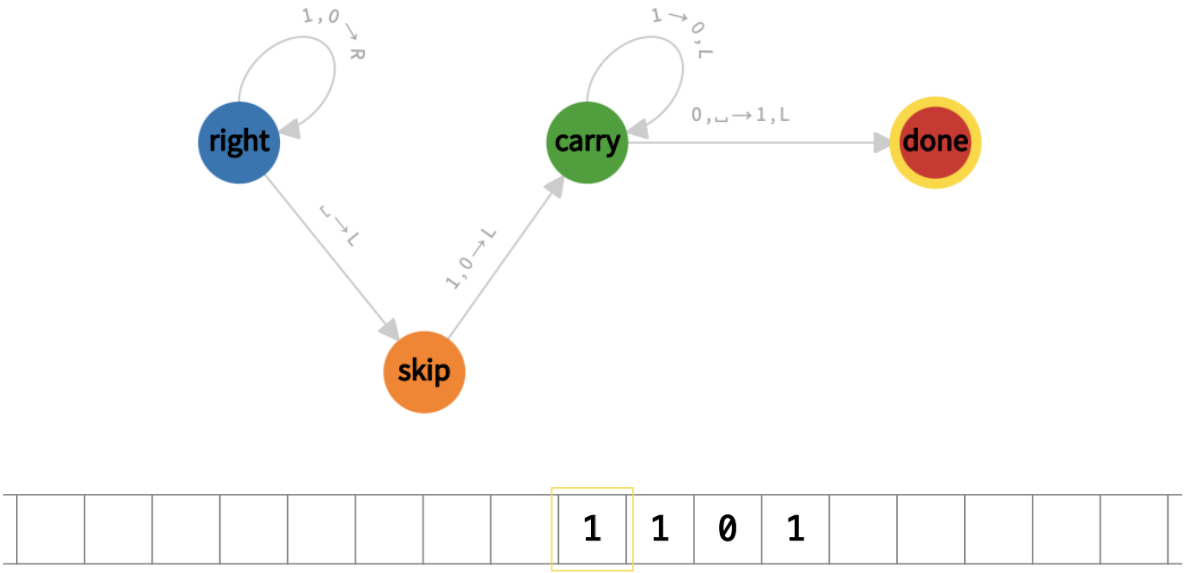
Therefore the DFA ends in the accepting state $(1, 1)$ exactly when $\#_0(w) \bmod 3 = 1$ and $\#_1(w) \bmod 2 = 1$, which is the language asked for. Checking the examples from the problem: 01 ends in $(1, 1)$, 01000 has four 0's and one 1 so it ends in $(1, 1)$, and 1110 has one 0 and three 1's so it ends in $(1, 1)$; while $\varepsilon \to (0, 0)$, $1 \to (0, 1)$, $001 \to (2, 1)$, $0011 \to (2, 0)$, and $000111 \to (0, 1)$ are all rejected.

Q5.1 Turing machines (a)

Adding 2 to a binary number is the same as adding 1 to the number with its last bit removed. So I keep the "binary increment" machine, but after scanning to the right end I step over the least significant bit before starting the carry. Here is the code I used on turingmachine.io (the only changes from the example are the new `skip` state and the transition into it):

```
# Adds 2 to a binary number.
input: '1011'
blank: ' '
start state: right
table:
  # scan to the rightmost digit
  right:
    [1,0]: R
    ' '  : {L: skip}
  # step over the least significant bit (it does not change)
  skip:
    [1,0]: {L: carry}
  # then carry the 1 into the remaining bits
  carry:
    1      : {write: 0, L}
    [0,' ']: {write: 1, L: done}
  done:
```

Here is the resulting machine on turingmachine.io, after running it on the input `1011`; the tape now reads `1101`.

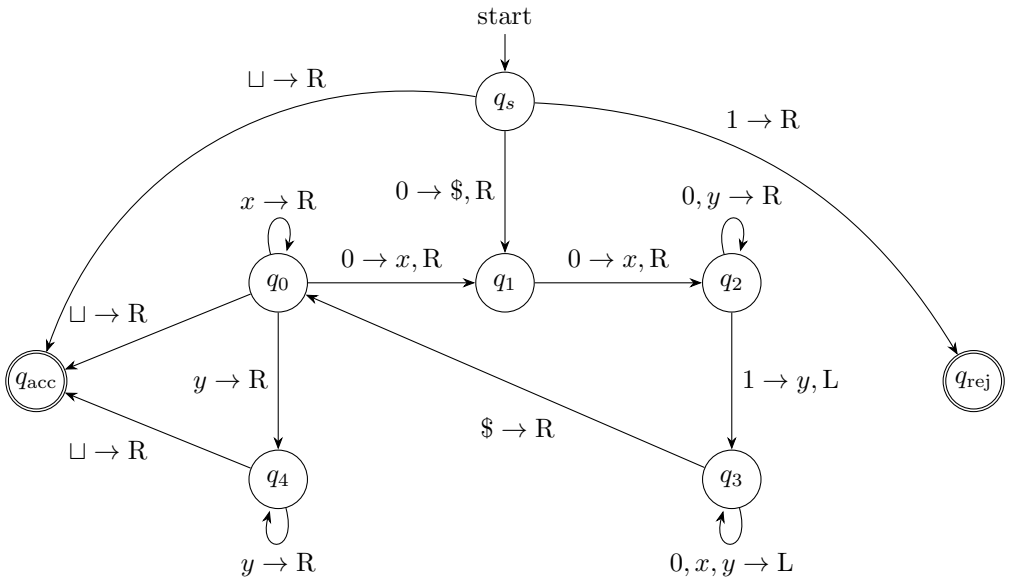


For example, on input `1011` (eleven) the machine scans right to the blank, steps left onto the final `1`, steps left again onto `1` in state `carry`, writes `0` and moves left, reads `0`, writes `1` and stops in `done`. The tape now reads `1101` (thirteen). On input `1` the carry runs into the blank to the left and writes a `1` there, giving `11` (three).

Q5.2 Turing machines (b)

Idea. Repeatedly cross off two 0's from the left and one 1 from the left end of the 1's. Crossed-off 0's are overwritten with x and crossed-off 1's with y. The very first 0 is overwritten with \$ instead of x so that the head can find the left end of the tape when it returns. If a round cannot find its two 0's and one 1 in the right order, reject. If the head scans the whole tape and finds only crossed-off symbols, accept.

Tape alphabet $\Gamma = \{0, 1, x, y, \$, \sqcup\}$, input alphabet $\{0, 1\}$. Edge labels are *read* $\to$ *write*, *move*; when the write is omitted the symbol is left unchanged. Every transition not drawn goes to q_{rej} (moving right, say).



In words, the states mean:

- q_s (start): if the tape is empty, accept ($k = 0$); if it starts with 0, mark it with \$ and look for the second 0; if it starts with 1, reject.
- q_0: skip crossed-off 0's (x) and find the next uncrossed 0. If instead we see y, all 0's are used up, so go to q_4 to check that nothing but crossed-off 1's remain. If we see a blank, everything has been matched: accept. Seeing a 1 means an unmatched 1 with no 0's left: reject.
- q_1: the second 0 of this round must come immediately; cross it off. Anything else means the number of 0's is odd or the string is out of order: reject.
- q_2: move right over the remaining 0's and already-crossed 1's (y) to the first uncrossed 1, cross it off, and turn around. A blank here means there are not enough 1's: reject.
- q_3: move left back to the \$ marker, then start the next round in q_0.
- q_4: after the last 0 was used, the rest of the tape must consist only of y's up to the blank; then accept. Any 0 or 1 here means a leftover or out-of-order symbol: reject.

Each round removes exactly two 0's and one 1, the 0's are always taken from the left and the 1 from the left end of the 1's, and the machine only accepts when every symbol has been crossed off with the tape in the form $\$\, x^* \, y^*$. So it accepts exactly the strings $0^{2k}1^k$, and it halts on every input because every round moves the head strictly further into the input.

Q6.1 Decidability (a)

Let $L = \{\ell_1, \ldots, \ell_n\}$ be a finite language. The strings $\ell_1, \ldots, \ell_n$ are fixed, so they can be written directly into the algorithm.

Input: A string x
Output: accept if $x \in L$, reject otherwise

```
1: function DecideFinite(x)
2:     for i = 1, ..., n do
3:         if x = ℓ_i then                          ▷ compare character by character
4:             return accept
5:         end if
6:     end for
7:     return reject
8: end function
```

It always halts. The loop runs at most n times, and n is a fixed constant that does not depend on x. Each iteration compares two finite strings, which takes at most $\max(|x|, |\ell_i|) + 1$ steps. So the algorithm halts on every input.

It is correct. If $x \in L$, then $x = \ell_i$ for some i, so the loop reaches that i (or an earlier match) and returns accept. If $x \notin L$, then $x \neq \ell_i$ for every i, so no iteration returns, and the algorithm returns reject after the loop. So the algorithm accepts exactly the strings in L.

Since there is an algorithm that halts on every input and returns the right answer, and every algorithm can be implemented by a Turing machine, L is decidable. (The case $n = 0$, the empty language, is also covered: the loop body never runs and the algorithm always rejects.)

Q6.2 Decidability (b)

Since L_1, L_2, L_3 are decidable, there are algorithms D_1, D_2, D_3 that halt on every input and satisfy $D_i(x) =$ accept if and only if $x \in L_i$. I use them as subroutines.

Input: A string x
Output: accept if $x \in (L_1 \setminus L_2) \cup L_3$, reject otherwise

```
1: function DecideL(x)
2:     if D_3(x) = accept then
3:         return accept
4:     end if
5:     if D_1(x) = accept and D_2(x) = reject then
6:         return accept
7:     end if
8:     return reject
9: end function
```

Halting. The algorithm makes at most three calls, to D_3, D_1, and D_2, each of which halts on every input because the L_i are decidable. Everything else is a constant number of comparisons. So DecideL halts on every input.

Correctness. Spelling out the definition, $x \in L$ if and only if $x \in L_3$, or ($x \in L_1$ and $x \notin L_2$).

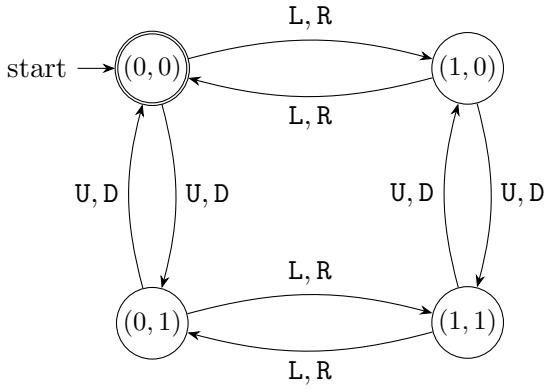
- If $x \in L$ and $x \in L_3$, then $D_3(x)$ accepts, so the algorithm accepts at the first **if**.
- If $x \in L$ and $x \notin L_3$, then $x \in L_1$ and $x \notin L_2$, so $D_1(x)$ accepts and $D_2(x)$ rejects, and the algorithm accepts at the second **if**.
- If $x \notin L$, then $x \notin L_3$, so the first **if** does not fire, and it is not the case that ($x \in L_1$ and $x \notin L_2$), so the second **if** does not fire either. The algorithm reaches the last line and rejects.

So DecideL accepts exactly the strings in L, and L is decidable.

Q7.1 Pac-Man's journey (a)

Pac-Man's final position is $(\#\mathtt{R} - \#\mathtt{L},\ \#\mathtt{U} - \#\mathtt{D})$. Whether a coordinate is even depends only on its value mod 2, and $\#\mathtt{R} - \#\mathtt{L} \equiv \#\mathtt{R} + \#\mathtt{L} \pmod 2$. So the DFA only needs to track the parity of the x-coordinate and the parity of the y-coordinate. Each L or R flips the x-parity, and each U or D flips the y-parity.

States are pairs $(p_x, p_y) \in \{0,1\}^2$, where p_x is the x-coordinate mod 2 and p_y is the y-coordinate mod 2.



Formally: $\Sigma = \{\mathtt{L}, \mathtt{R}, \mathtt{U}, \mathtt{D}\}$, $Q = \{0,1\}^2$, start state $(0,0)$, $F = \{(0,0)\}$, and

$$\delta\big((p_x, p_y), \mathtt{L}\big) = \delta\big((p_x, p_y), \mathtt{R}\big) = (1 - p_x,\ p_y), \qquad \delta\big((p_x, p_y), \mathtt{U}\big) = \delta\big((p_x, p_y), \mathtt{D}\big) = (p_x,\ 1 - p_y).$$

Justification. By induction on the length of the command string s, after reading s the DFA is in state ($x \bmod 2,\ y \bmod 2$) where (x, y) is Pac-Man's position after executing s. This is true for $s = \varepsilon$ since Pac-Man starts at $(0,0)$ and so does the DFA. Appending one command changes exactly one coordinate by ± 1, which flips that coordinate's parity and leaves the other alone, and that is exactly what the corresponding transition does. Pac-Man ends at an even position exactly when both parities are 0, i.e., when the DFA ends in $(0,0)$, the only accepting state.

Q7.2 Pac-Man's journey (b)

Let L_{home} be the set of command strings after which Pac-Man is back at $(0, 0)$. Suppose, for contradiction, that some DFA M with p states decides L_{home}.

Consider the $p + 1$ strings $\mathtt{R}^0, \mathtt{R}^1, \mathtt{R}^2, \ldots, \mathtt{R}^p$. Since M has only p states, by the pigeonhole principle two of these strings, say $\mathtt{R}^i$ and $\mathtt{R}^j$ with $0 \le i < j \le p$, leave M in the same state q.

A DFA's future behavior depends only on its current state and the remaining input. So for any string z, the strings $\mathtt{R}^i z$ and $\mathtt{R}^j z$ end in the same state, and M either accepts both or rejects both.

Now take $z = \mathtt{L}^i$.

- $\mathtt{R}^i\mathtt{L}^i$ moves Pac-Man i steps right and then i steps left, ending at $(0, 0)$. So it is in L_{home}, and M must accept it.
- $\mathtt{R}^j\mathtt{L}^i$ ends at $(j - i, 0)$ with $j - i \ge 1$, which is not the origin. So it is not in L_{home}, and M must reject it.

But M treats both strings the same way, so it gives the wrong answer on one of them. This contradicts the assumption that M decides L_{home}. Since p was arbitrary, no DFA decides L_{home}.

The intuition is just that a DFA has finitely many states, but to know whether Pac-Man gets home it would need to remember how far right it has gone, and that can be any integer.

Q7.3 Pac-Man's journey (c)

Input: A command string $s = s_1 s_2 \cdots s_n$ over $\{\texttt{L}, \texttt{R}, \texttt{U}, \texttt{D}\}$
Output: accept if Pac-Man ends at $(0, 0)$ after executing s, reject otherwise

```
1: function BackHome(s)
2:     x ← 0, y ← 0
3:     for i = 1, ..., n do
4:         if s_i = L then x ← x − 1
5:         else if s_i = R then x ← x + 1
6:         else if s_i = U then y ← y + 1
7:         else y ← y − 1                                   ▷ s_i = D
8:         end if
9:     end for
10:    if x = 0 and y = 0 then return accept
11:    else return reject
12:    end if
13: end function
```

Correctness. Claim: after the i-th iteration of the loop, (x, y) equals Pac-Man's position after executing the first i commands $s_1 \cdots s_i$. For $i = 0$ both are $(0, 0)$, the starting position. If the claim holds after iteration $i - 1$, then the i-th iteration changes (x, y) by exactly the unit move that command s_i makes (left decreases x, right increases x, up increases y, down decreases y), so the claim holds after iteration i. By induction, after the loop (x, y) is Pac-Man's final position, and the algorithm accepts exactly when that position is $(0, 0)$.

The loop runs n times, so the algorithm halts on every input, and this is a decider for L_{home}. The difference from a DFA is that the counters x and y can grow without bound, which a finite set of states cannot do.

Q8.3 Decision problems and languages (c)

"Given an array S of integers, are all its elements distinct?"

(i) The elements can be negative, so I add a minus sign to the alphabet from the sorted-array example: $\Sigma = \{\texttt{0}, \ldots, \texttt{9}, -, [, ,,]\}$.

(ii) Encoding: write the elements in base 10 (with a leading $-$ for negative numbers), separated by commas and enclosed in brackets. For example, the array S with entries five, negative twelve, zero, and five has $\langle S \rangle = [\texttt{5}, -\texttt{12}, \texttt{0}, \texttt{5}]$.

(iii) For $S = (a_1, \ldots, a_n)$, element a_i takes $1 + \lfloor \log_{10} |a_i| \rfloor$ digits (one digit when $a_i = 0$), plus one more character if it is negative, so $\Theta(\log(|a_i| + 2))$ characters. Add the two brackets and $n - 1$ commas, which is $n + 1$ characters. The total length is

$$|\langle S \rangle| = \Theta\Big(n + \sum_{i=1}^{n} \log(|a_i| + 2)\Big).$$

If every element has absolute value at most M, this is $O(n \log M)$.

(iv) The language is

$$L_{\text{distinct}} = \{\langle S \rangle : S \text{ is an array of integers and } a_i \neq a_j \text{ for all } i \neq j\}.$$

Q8.4 Decision problems and languages (d)

"Is a given undirected graph complete?"

(i) Number the vertices $1, \ldots, n$ and write numbers in base 10. $\Sigma = \{\texttt{0}, \ldots, \texttt{9}, (,), ,, ;\}$.

(ii) Encoding: write the number of vertices n, then a semicolon, then the list of edges, each as a pair (u,v) of vertex numbers, separated by commas. For example, the triangle on three vertices has

$$\langle G \rangle = \texttt{3};(\texttt{1},\texttt{2}),(\texttt{1},\texttt{3}),(\texttt{2},\texttt{3}),$$

and the path $1 - 2 - 3$ has $\langle G \rangle = \texttt{3};(\texttt{1},\texttt{2}),(\texttt{2},\texttt{3})$.

(iii) Let G have n vertices and m edges. The vertex count takes $\Theta(\log n)$ digits. Each edge takes two vertex numbers of at most $1 + \lfloor \log_{10} n \rfloor$ digits each, plus four punctuation characters, so $\Theta(\log n)$ characters (treating $\log 1$ as 1 character). The total length is

$$|\langle G \rangle| = \Theta\big((m+1)\log n\big).$$

Since $m \leq \binom{n}{2}$, this is $O(n^2 \log n)$ in the worst case. (An adjacency-matrix encoding over $\{\texttt{0}, \texttt{1}, ;\}$ would have length $\Theta(n^2)$ instead.)

(iv) The language is

$$L_{\text{complete}} = \{\langle G \rangle : G \text{ is a complete undirected graph}\},$$

where "complete" means that every two distinct vertices of G are adjacent. Equivalently, if G has n vertices, its edge set is exactly $\{(u,v) : 1 \leq u < v \leq n\}$.

Q8.5 Decision problems and languages (e)

"Does a given C++ program terminate when run on a given input?"

(i) A C++ source file is a string of ASCII characters, and so is the program's input. I take Σ to be the set of ASCII characters plus one extra separator symbol # that is not an ASCII character, so that it cannot appear inside the program or its input.

(ii) Encoding: the input object is a pair (program P, input string x). Encode it as

$$\langle P, x \rangle = \langle P \rangle \,\#\, \langle x \rangle,$$

where $\langle P \rangle$ is the source code of P written out character by character, and $\langle x \rangle$ is the input string itself. For example, if P is the program `int main(){while(1);}` and x is the input `42`, then $\langle P, x \rangle =$ `int main(){while(1);}#42`. Because # occurs exactly once, the encoding can be split back into P and x unambiguously.

(iii) The length is simply the length of the source code plus the length of the input plus one:

$$|\langle P, x \rangle| = |P| + |x| + 1 = \Theta(|P| + |x|).$$

(iv) The language is

$$L_{\text{halt}} = \{\langle P, x \rangle : P \text{ is a valid C++ program and } P \text{ halts when run on input } x\}.$$

(This is just the halting problem with C++ as the programming language. L_{halt} is a perfectly well-defined language even though no algorithm decides it.)

Q9.1 Trophy testing (a, optional)

Method. Suppose I have d drops left and both blocks intact. Drop the first block from height $h = d$.

- If it breaks, the LBH is one of $1, \ldots, d$. I have one block and $d-1$ drops left, so I test heights $1, 2, \ldots, d-1$ in order with the second block. The first height at which it breaks is the LBH; if it never breaks, the LBH is d. This never needs more than $d-1$ drops.
- If it does not break, the LBH is more than d. I still have two blocks and $d-1$ drops, so I recursively run the same strategy on the heights above d, i.e., the next drop is from height $d + (d-1)$, and so on.

In other words, the first-block drops are at heights d, $d + (d-1)$, $d + (d-1) + (d-2), \ldots$, and when the first block breaks, the second block sweeps the gap below it one inch at a time. After d drops, every gap has been searched or the total height has been cleared.

Recurrence. Let $n(d)$ be the largest n for which d drops and two blocks suffice to determine the LBH among the $n+1$ possibilities $\{1, \ldots, n, \text{“more than } n\text{”}\}$. With 0 drops nothing can be tested, so only one possibility can be handled: $n(0) = 0$. With $d \geq 1$ drops, the first drop from height d covers d heights (the heights $1, \ldots, d$ in the break case), and the no-break case leaves a fresh $(d-1)$-drop problem that covers $n(d-1)$ further heights. So

$$n(d) = d + n(d-1), \qquad n(0) = 0.$$

Closed form. Unrolling, $n(d) = d + (d-1) + \cdots + 1 + n(0)$, so

$$n(d) = \frac{d(d+1)}{2}.$$

As a check: $n(1) = 1$ (drop from 1: breaks means LBH 1, otherwise “more than 1”), $n(2) = 3$, $n(3) = 6$, $n(4) = 10$.

Q9.2 Trophy testing (b, optional)

Let $N(d)$ be the largest n that *any* strategy with two blocks and at most d drops can handle. I show $N(d) \le d + N(d-1)$ with $N(0) = 0$, which gives $N(d) \le d(d+1)/2 = n(d)$ by the same unrolling as in part (a).

With 0 drops there is no information, so only one possibility can be distinguished: $N(0) = 0$.

Now consider any strategy with $d \ge 1$ drops, and let h be the height of its first drop. I have to be ready for either outcome of that drop.

- **If it breaks**, the LBH is in $\{1, \dots, h\}$, and only one block remains. With one block, no height below h can be skipped: if the strategy skipped some height $t < h$ and the block then broke at a higher height, it could not tell whether the LBH was t or that higher height, and there are no blocks left to find out. So the strategy must drop from $1, 2, \dots$ one at a time. With $d-1$ drops left it can check at most $d-1$ heights, which decides among at most d possibilities ($1, \dots, d-1$ and "higher"), so $h \le d$.
- **If it does not break**, the LBH is more than h. The strategy now has two blocks and $d-1$ drops, so by definition of N it can handle at most $N(d-1)$ heights above h.

Putting the two cases together, the strategy can handle at most $h + N(d-1) \le d + N(d-1)$ heights. Since this holds for every strategy, $N(d) \le d + N(d-1)$, and so $N(d) \le d(d+1)/2 = n(d)$. The strategy in part (a) achieves this bound, so it is optimal.